规培

September 2026

1 概念扫盲

1.1 各个模块

1.1.1 PS

Processing System

处理系统, 即 Zynq 芯片中的 ARM 处理器部分。包含双核 Cortex-A9、内存控制器、外设接口（UART、SPI、I2C、以太网等）。PS 是” 软件侧”, 运行 C 语言程序。

1.1.2 PL

Programmable Logic

可编程逻辑, 即 Zynq 芯片中的 FPGA 部分。基于 Artix-7 架构, 可以用 Verilog/VHDL 编程实现任意数字电路。PL 是” 硬件侧”, 实现自定义逻辑。

1.1.3 AXI

Advanced eXtensible Interface

ARM 公司提出的片上总线协议, 是 AMBA 3.0/4.0 标准的一部分。分为 AXI-Full（高性能数据传输）、AXI-Lite（轻量寄存器访问）、AXI-Stream（流式数据）三种类型。Zynq 中 PS 与 PL 的主要通信通道都基于 AXI。

1.1.4 AXI-Lite

Advanced eXtensible Interface Lite

AXI 总线的轻量子集, 是 PS 与 PL 之间最常用的通信接口之一。采用寄存器映射方式,PS 端通过读写 PL 端的寄存器来控制硬件逻辑。适合低速、控制类应用, 如 LED 控制、寄存器配置等。

详细介绍见文档《ZYNQ-以规培项目为例》。

1.1.5 Shell

一种通过串口与用户交互的命令行界面。用户输入命令字符串, 系统解析并执行对应操作, 再通过串口返回结果。类似 Windows 的 CMD 或 Linux 的 Terminal, 但运行在嵌入式系统上。

1.1.6 Block Design

Vivado 中的图形化 IP 集成方式。通过拖拽和连线的方式, 将 Zynq PS、AXI 接口、自定义 IP 等模块连接起来, 构建完整的硬件系统。是 Zynq 开发中连接 PS 和 PL 的主要方式。

1.2 PS-PL 架构与数据流

本任务的核心是 PS 通过 AXI-Lite 总线控制 PL 端的 LED。理解数据如何从用户输入最终流向 LED 是关键。

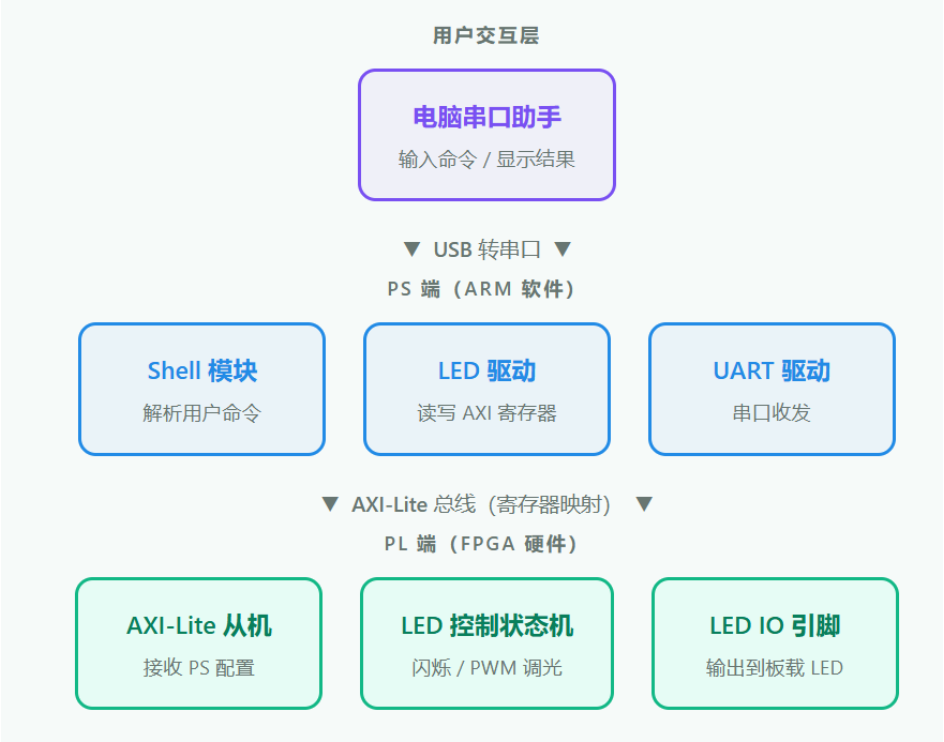


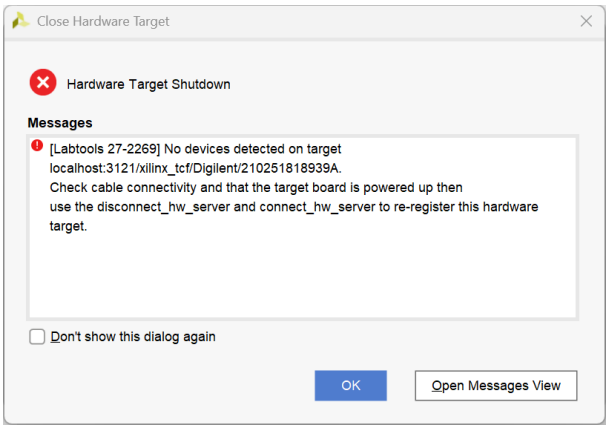
图 1: 整体架构

整个数据流可以概括为：用户在电脑上通过串口助手输入命令 → PS 端的 Shell 解析命令 → LED 驱动通过 AXI-Lite 总线写入 PL 端的控制寄存器 → PL 端的状态机根据寄存器值改变 LED 行为 → LED 引脚输出相应的电平或 PWM 信号 → LED 呈现出闪烁或调光效果。

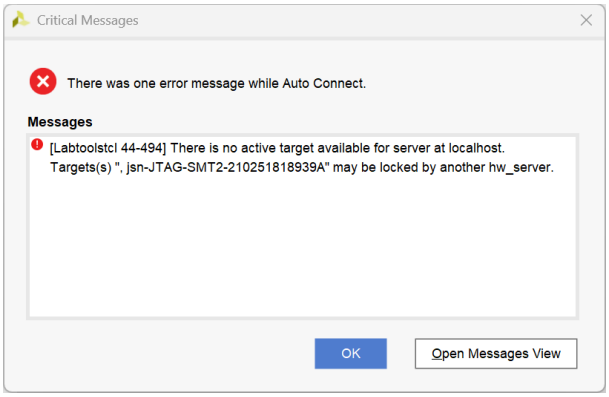
这部分详细可看《ZYNQ 硬件-以规培为例》。

2 硬件问题

使用板子时要解决三个问题, 分别关于电源、串口、下载器。这三个部分会影响 FPGA 的连接, 如有不慎会出现以下报错:



(a) 连接错误



(b) JTAG 未连接

图 2: 位置确定

2.1 电源

板子在 J4 接口处有专门的电源供电, 需要买电源适配器。使用 6pin 接口, 注意板子背面 D24 需要导通, 需要焊接。

另附：在尝试过程使用 20pin 的引脚进行供电, 那么此时需要 DCDC 进行电源供电。为了满足其供电需求,12V 和 400mA 以上,DCDC 模块的电源适配器选择应是 12V/2A, 保证输出的电压可以供板子。(不建议使用后者, 使用前者)

2.2 串口

在板子买回来后, 串口排针 J7 是空的, 需要进行焊接。在调试过程中需要用到串口调试，检查输出；最终结果时需要将串口连接电脑在 SDK Terminal 中输入命令控制。

2.3 下载器

不同于 typeC 接口, 该板子需要通过 JTAG 口进行烧录代码。

先在板子 J8 的接口将 14pin 的排针焊接完, 买下载器, 将排线和接口对齐接上。注意下载器的排线两头间距要适配焊的排针间距 (建议都用 2.54mm), 下载器注意 GND 和 Vref 不要接反, 会烧板。

初次上电后有三个灯亮表示正常：电源灯 LED4 和芯片灯 LED1 都会亮，同时板子的 PWM 灯也会亮，绿灯亮，红灯等一段时间闪烁。

3 注意事项

3.1 大致流程

在进行搭建过程时若时间充裕, 推荐如下流程:

1. 先在 vivado 平台搭建硬件，此时只加入 PS 来进行处理。这里简单设置串口，结合原理图，MIO 口是 24-25，这里配错也会导致烧录失败。在 SDK（这里 vivado 选择 18.3）里选择简单的 Hello World 代码用于串口测试，在 SDK Terminal 里查看到输出的结果即可。
2. 尝试点灯，这里对灯做一个选择，有 VGA 转接板可以选择 J18 和 H15，没有转接板就选择板子上的红灯 W13（尝试 ing）。
3. 若要进行 shell 移植，选择开源代码或实验室代码均可。先从简单命令了解大致流程后上升工程使用。(这部分检查类似于串口调试，发送和返回情况的日志)
4. 结合要求写代码，可以问 AI，但是不能代替工作，推荐古法编程。

3.2 手搓代码

关于整个流程中需要搞明白:

1. shell 的工作流程：其中要用什么函数？自己的读写函数怎么写？命令设定是什么（保证命令被识别)?
2. FPGA 部分：有没有给 AXI 总线接口设置地址?（这里以灯控代码写在 PL 端为例）灯控.v 文件是否需要放进.bd 图里一起打包？要实现 AXI 总线通信都需要添加哪些模块，模块和模块之间怎么接线?
3. SDK 部分：给 shellCommand 内存布局了吗（即在.ld 文件里是否添加起始地址索引和终点地址索引)？移植后有没有需要修改的参数？读函数中怎么判断 FIFO 里有没有数据？写函数里怎么判断 FIFO 里有没有空位，想要完成这两项功能要看什么文件？对于读写寄存器的位置是选择写死还是用结构体表示?

   ...

4 移植 shell

以移植 GitHub 开源 shell 为例，移植版本 3.2.4.

用户输入命令 -> 通过 read/write 函数写数据 -> 分别在 task 函数里解析和处理命令和数据 -> 观察结果

4.1 命令名

这就意味着输入的命令需要名字，文件 shell.h 中：

```
#define SHELL_EXPORT_CMD(_attr, _name, _func, _desc)
```

这个宏定义在给自己建立的命令进行设置并纳入 shellCommandList 中。这四个参数的意思分别是：

表 1: 参数解释

参数	名字	类型	作用	举例
参数 1	`_attr`	属性（数字）	命令的权限、类型等属性	`SHELL_CMD_PERMISSION() | SHELL_CMD_TYPE(SHELL_TYPE_CMD_MAIN)`
参数 2	`_name`	标识符	命令名（你在串口里输入的名字）	`led`
参数 3	`_func`	函数指针	对应的 C 函数名（执行命令时调用这个函数）	`led_cmd`
参数 4	`_desc`	字符串	命令描述（help 命令时显示的帮助文字）	“LED control command”

4.2 读函数

在 shell.h 文件里提到了读写函数设置规则:

```
signed short (*read)(char *, unsigned short);
signed short (*write)(char *, unsigned short);
```

在此引入两个参量，即数据指针和字符长度，通过 xuartps.h 文件中的函数去进行数据读取，这里需要设置读取地址，推荐使用结构体，这样可以在换平台是由运行 bsp 自动获取，防止出错。每次读取完要返回 1，没有读取到返回 0。

4.3 写函数

设置原理同上。

4.4 main 函数设置

和 STM 中初始化很像，对 shell 进行初始化，调用函数；对串口进行初始化，这里函数需要获取设备地址，可以用带有的 Lookup 函数进行查找。while 里需要包括 shell 处理函数。

5 硬件平台配置

详细见《ZYNQ-以规培为例》。